# Lab Session – Federated Learning with GPU-Accelerated Services (Bonus Session)

## Learning Objectives

By the end of this lab, you will:

1. Understand the **concept and architecture of Federated Learning (FL)**.
2. Implement a **simple federated learning simulation** using **Numba** for GPU acceleration.
3. Build **local training services** that return model updates.
4. Implement a **central aggregator service** that produces global model parameters.
5. Observe GPU-accelerated computation in a distributed learning workflow.

---

## Context

Federated Learning (FL) enables multiple clients to train a shared machine-learning model **without sharing their raw data**. Each client trains a local model and only sends **model parameters** to a central **aggregator**.

In this lab, you will simulate this setup using:

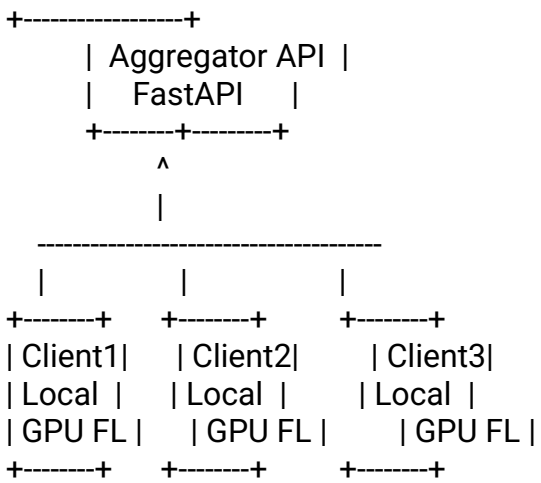
- **Three client services** (local training + GPU acceleration)
- **One aggregator service** (model averaging)
- **FastAPI** microservices
- **Numba GPU kernels**

---

## Federated Learning Setup Overview

| Component | Description |
|---|---|
| **Client Service (×3)** | Performs local training on GPU and returns learned weights. |
| **Aggregator Service** | Collects local updates, computes global average, and returns global model. |
| **Communication** | JSON over HTTP (FastAPI). |
| **Model** | Linear regression: y = w * x + b. |
| **GPU Use** | Numba CUDA kernel for gradient computation. |

# Architecture Diagram




---

# TASK 1 — Implement a Local Training Service

Each client must expose:

| Endpoint | Description |
|---|---|
| POST /train | Runs local training and returns {w, b}. |
| GET /health | Returns service health status. |

---

# Task 1.1 — Local GPU Training with Numba

Each client should:

1. Generate synthetic data
2. Train a simple linear regression model
3. Implement gradient computation using a **Numba CUDA kernel**
4. Return the locally trained weights

A skeleton of the kernel and API structure is provided in the lab PDF (students must complete it).

---

# TASK 2 — Implement the Aggregator Service

The aggregator must expose:

| Endpoint | Description |
|---|---|
| POST /aggregate | Receives local client weights and returns the global aggregated weights. |
| GET /health | Returns service health status. |

The aggregator should compute:

- Global **w** = average of client w values
- Global **b** = average of client b values

---

# TASK 3 — Simulate a Federated Learning Round

You will:

1. Run 3 client services
2. Run the aggregator
3. Call `/train` on each client
4. Send results to `/aggregate`
5. Inspect the aggregated global model parameters

Suggested local testing commands (adapt to your environment):

```
uvicorn client1:app --port 8001
uvicorn client2:app --port 8002
uvicorn client3:app --port 8003

uvicorn aggregator:app --port 9000
```

Then:

```
curl -X POST http://localhost:8001/train
curl -X POST http://localhost:8002/train
curl -X POST http://localhost:8003/train
```

Finally call the aggregator with the three results.

---

# TASK 4 — Deploy All Services with Docker Compose

Create a **docker-compose.yml** that runs:

- 3 GPU-enabled client containers
- 1 aggregator container

Use the structure provided in the lab description to complete your file.

Launch all services using:

```
docker compose up --build
```

---

# TASK 5 — Observe & Discuss

During your experiment:

## 1. Compare

- Client training results
- Aggregated global model

## 2. Reflect

- How close the global model is to the ideal theoretical values (`w=3.5`, `b=2.0`)
- Whether the GPU is being used during computation

## 3. Monitor

Run this to observe GPU activity:

```
watch nvidia-smi
```

---

# BONUS CHALLENGES (OPTIONAL)

- Add **multiple federated rounds**.
- Implement **logistic regression** instead of linear regression.
- Add **Prometheus metrics** to monitor GPU utilization.
- Visualize convergence curves in **Grafana**.

---

# Deliverables

1. Source code (published on Github) of: * 3 client services * 1 aggregator service
2. Complete `docker-compose.yml`

## Important Note

Once you finish the entire lab, **contact the instructor to receive access to a remote server** where you will simulate the complete Federated Learning environment.

---

# Annexe

## Allowed Libraries

For this lab, you **must NOT use** any deep learning framework, including:

- PyTorch
- TensorFlow
- JAX
- Keras
- Flax
- MXNet
- Any auto-differentiation or high-level ML library

You may only use:

- **NumPy** □ for CPU-side numerical operations
- **Numba (CUDA)** □ for GPU-side kernels and accelerated computation
- **FastAPI** □ for the microservices
- **Standard Python libraries**

---

# Federated Learning Principles

Federated Learning (FL) is a distributed machine-learning approach where **multiple clients train a shared global model collaboratively** without sending their raw data to a central server.

Instead of centralizing data, FL centralizes **model updates**.

## 1. Local Training on Each Client

Each client (device, service, or edge node):

1. Keeps its private dataset locally
2. Trains a **local model** on this private data
3. Computes local model parameters $(w, b)$ or gradients
4. Sends only the model updates to the aggregator, *not the dataset*

This improves privacy because raw data never leaves the device.

---

## 2. The Aggregator Computes a Global Model

The central server (aggregator):

1. Receives updates from all clients
2. Computes the **global average** of these parameters
3. Produces a new global model
4. Optionally sends the global model back to clients in multi-round FL

In this lab, you will focus on **one-round federated averaging**:

$$w_{global} = \frac{1}{K} \sum_{k=1}^{K} w_k$$

$$b_{global} = \frac{1}{K} \sum_{k=1}^{K} b_k$$

Where

$K$

= number of clients.

---

# 3. Privacy Advantage

Because each client only sends:

- A few numbers (model parameters)
- Not the raw training data

FL supports:

- **Data privacy**
- **Compliance with sensitive data regulations**
- **Distributed computation at the edge**

---

# 4. Communication Efficiency

FL reduces the need to transmit entire datasets. Instead, only **lightweight model updates** are exchanged.

This is especially useful in:

- Mobile devices
- IoT sensors
- Distributed enterprise applications

---

# 5. FL in a Service-Oriented Architecture (SOA)

This lab simulates FL using **microservices**:

- Each client is an independent service exposing `/train`
- The aggregator is another service exposing `/aggregate`
- Communication uses **HTTP + JSON**
- Each component is loosely coupled and independently deployable

This demonstrates how FL naturally fits the SOA paradigm.

---

## 6. FL Rounds (Optional Extension)

More advanced FL systems perform multiple rounds:

1. Clients receive global model
2. They train locally again
3. They send new updates
4. Aggregator updates global model

This continues until convergence.

# Mathematical Model to Implement

Each client trains a **simple linear regression model** of the form:

$$y = wx + b$$

Given a dataset

$(x_i, y_i)$

, you will compute the gradients manually (no autograd):

## 1. Prediction

$$\hat{y}_i = wx_i + b$$

## 2. Error

$$e_i = \hat{y}_i - y_i$$

## 3. Gradients

The gradients of the Mean Squared Error are:

$$\nabla_w = \frac{1}{N} \sum_{i=1}^{N} e_i \cdot x_i$$

$$\nabla_b = \frac{1}{N} \sum_{i=1}^{N} e_i$$

You must compute these **on the GPU** using a Numba CUDA kernel that:

- Computes local per-thread contributions
- Uses atomic operations to sum gradients safely

# 4. Parameter Update (SGD)

$$w := w - \eta, \nabla_w$$

$$b := b - \eta, \nabla_b$$

Where:

- $\eta$ = learning rate
- Updates must be applied on the **CPU** after copying gradients back from GPU memory